

Exercise: Practicing Exception Testing

- Now that exception testing with JUnit is understood, it's time to practice.
- The exercise focuses on adding a new method to the `Calculator` class.
- You will then create a corresponding unit test to validate exception behavior.
- The goal is to reinforce how `assertThrows` works in real test scenarios.

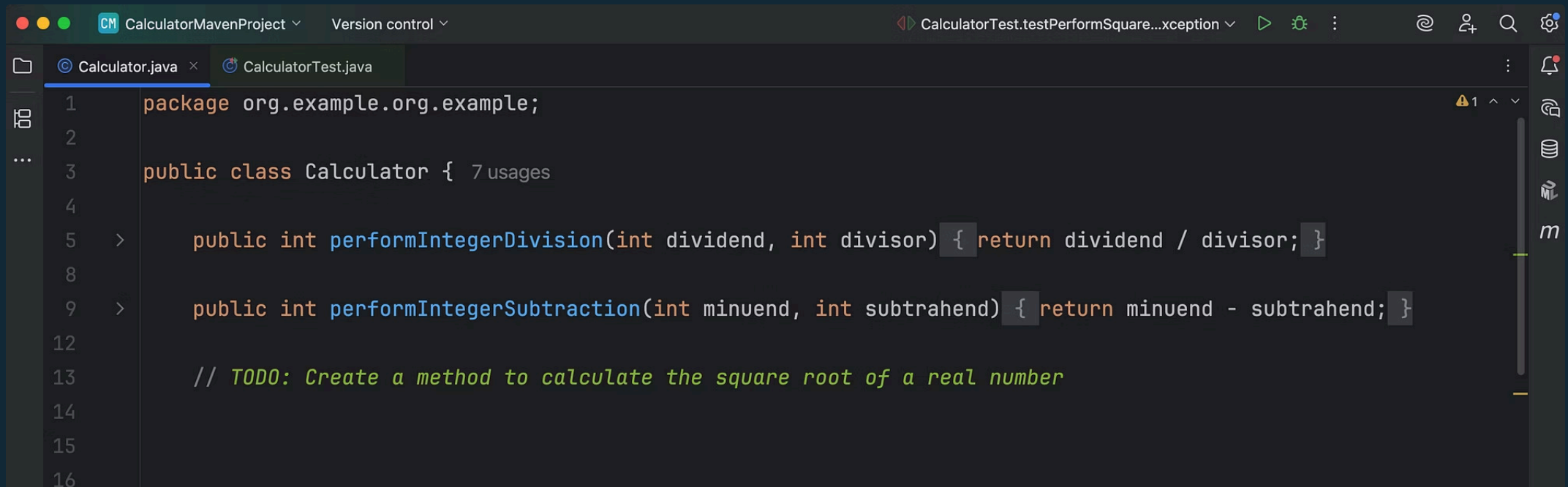
 by Rodrigo Martins Pagliares

Last update: 11/27/2025



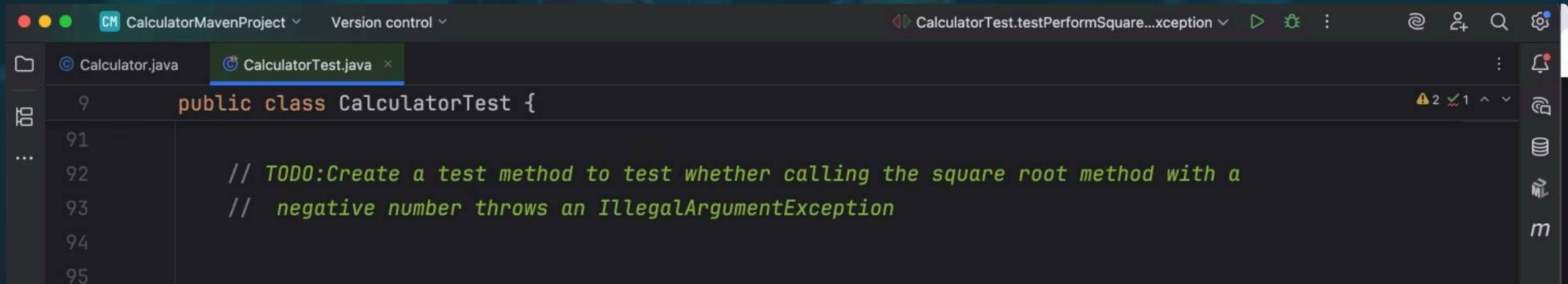
Exercise – Part 1 - Implementing the Square Root Method in Calculator Class

- Open your `Calculator` class and add a method to compute the square root of a real number.
- The method should follow a simple structure, similar to previous examples.
- This new method will be the **method under test** in the next step.
- Its behavior with invalid inputs will be the focus of your unit testing practice.



```
1 package org.example.org.example;
2
3 public class Calculator { 7 usages
4
5     > public int performIntegerDivision(int dividend, int divisor) { return dividend / divisor; }
6
7
8     > public int performIntegerSubtraction(int minuend, int subtrahend) { return minuend - subtrahend; }
9
10
11
12
13 // TODO: Create a method to calculate the square root of a real number
14
15
16
```

Exercise – Part 2 - Writing the Exception Test



```
CalculatorMavenProject Version control CalculatorTest.testPerformSquare...xception
Calculator.java CalculatorTest.java
9 public class CalculatorTest {
91
92 // TODO: Create a test method to test whether calling the square root method with a
93 // negative number throws an IllegalArgumentException
94
95
```

01

Open your CalculatorTest class and add a new test method.

02

The test should verify that calling the square root method with a **negative number** throws an `IllegalArgumentException`.

03

Use JUnit's exception-testing tools to validate this behavior.

04

After writing the test, run it to ensure it works as expected.

Conclusion

Strengthen Your Skills

This exercise strengthens your ability to test exception-throwing behavior in JUnit.

Mirror Real Workflows

Adding the square root method and its corresponding test mirrors real development workflows.

Prepare for More

Practicing both implementation and validation prepares you for larger testing scenarios.





Open Floor Discussion

- What additional edge cases could the square root method handle?
- Should the method return a special value instead of throwing an exception?
- How would you extend this test if more mathematical operations were added?